BéninConnect

Dossier technique architecture, structure et configuration

Application communautaire de la diaspora béninoise : application mobile iOS et Android, API .NET en monolithe modulaire, console d'administration, site public et infrastructure de production.

Éditeur : HBA Tech et Trade · Auteur : Hector Adjakpa
Version 1.0 du document · 8 novembre 2026
Domaines : monbenin.com · api.monbenin.com

Sommaire

1 Présentation du projet
2 Vue d'ensemble de l'architecture
3 Structure du dépôt
4 Backend : monolithe modulaire
5 Modules métier et API
6 Communication entre modules
7 Sécurité
8 Temps réel et notifications
9 Médias et stockage
10 Application mobile
11 Console d'administration et site public
12 Configuration
13 Infrastructure et déploiement
14 CI/CD
15 Tests et qualité
16 Publication sur les stores
17 Points restants et évolutions

Le projet en chiffres

9 modules backend	**~94** routes HTTP + 1 hub temps réel	**8** schémas PostgreSQL	**4** applications : mobile, API, console, site
66 tests backend (dont 2 S3)	**45** tests mobile (Jest)	**.NET 10** API	**Expo 58** React Native 0.88

> Ce document décrit l'état du projet au 8 novembre 2026 : code, configuration et infrastructure de production. Les secrets (clés, mots de passe, jetons) n'y figurent jamais : ils vivent uniquement dans le fichier `.env` du serveur, les secrets GitHub et les identifiants EAS.

1. Présentation du projet

BéninConnect réunit les Béninoises et les Béninois, où qu'ils vivent (Paris, Lyon, Bruxelles, Montréal, Cotonou…). L'application permet de retrouver les membres de sa ville, de partager des nouvelles, de découvrir les événements de la communauté et d'échanger en privé.

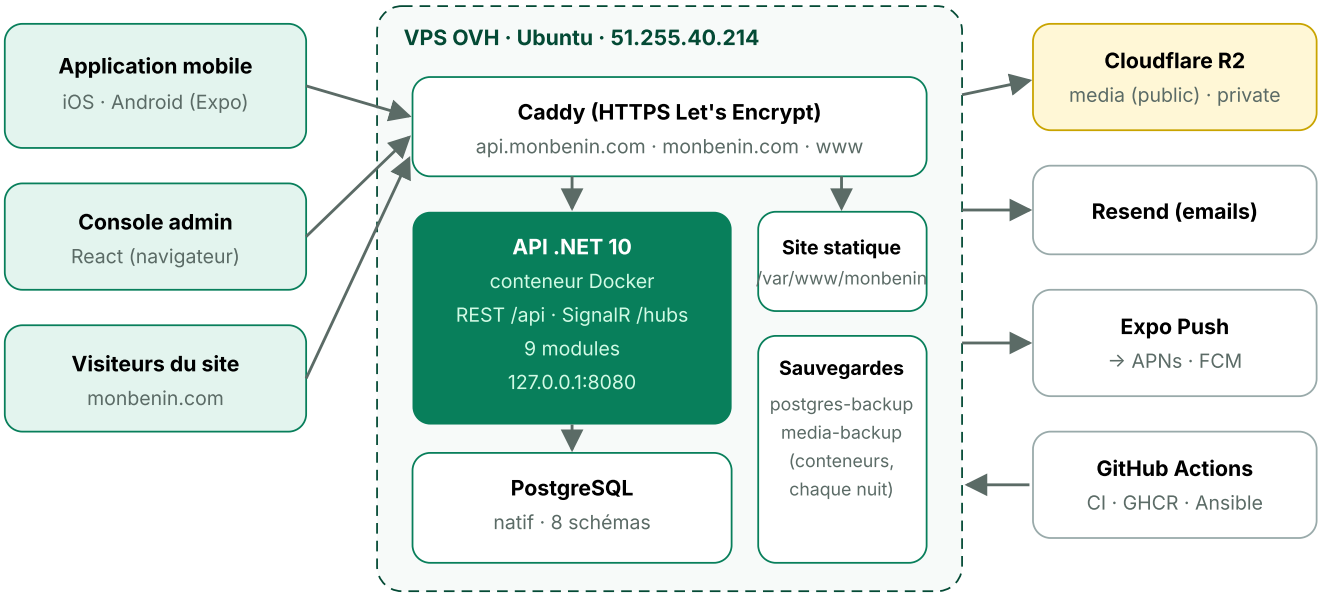
Fonctionnalités

Domaine	Fonctionnalités
Compte	Inscription avec code de vérification par email (Resend), connexion, mot de passe oublié, profil (photo, couverture, profession, ville, centres d'intérêt), confidentialité, **suppression du compte** dans l'application
Communautés	Communautés publiques ou privées (demandes d'adhésion), catégories, membres, création
Fil d'actualité	Publications texte, **photo ou vidéo** (60 s), visibilité (publique, communauté, moi seul), j'aime, commentaires, réactions emoji, partage natif, **modification et suppression** par l'auteur
Événements	Création, filtres (ville, catégorie, période), inscription, capacité
Messagerie	Conversations privées **en temps réel** (SignalR), réactions, accusés de lecture, **modification (15 min) et suppression** des messages, **pièces jointes** : photos, PDF, messages vocaux
Notifications	Centre de notifications, badge, **notifications push** iOS (APNs) et Android (FCM) via Expo, catégories réglables
Sécurité de la communauté	Signalement des publications, commentaires, membres et communautés ; modération (masquage, suppression, suspension) dans la console

Composants livrés

Composant	Technologie	Rôle
Application mobile	Expo 58, React Native 0.88, TypeScript	Application iOS et Android des membres (`mobile-expo/`)
API	ASP.NET Core .NET 10, EF Core, PostgreSQL	Monolithe modulaire, REST + SignalR (`server/`)
Console d'administration	React 19, Vite, TanStack Query	Modération, membres, statistiques, audit (`admin-console/`)
Site public	HTML/CSS statique	Accueil, support, confidentialité, conditions, mentions légales, suppression de compte (`site/`)
Prototype historique	Avalonia (.NET 9)	Première maquette hors ligne (`src/` , `tests/`), exclue de la CI

2. Vue d'ensemble de l'architecture



Flux principaux (les flèches vers l'extérieur partent de l'API)

1. Les clients appellent https://api.monbenin.com ; Caddy termine le TLS et relaie vers l'API (127.0.0.1:8080), WebSocket compris.
2. L'API lit et écrit dans PostgreSQL (un schéma et un DbContext par module) et range les fichiers dans R2 (ou le disque).
3. Les photos et vidéos publiques sont servies via l'URL R2 (redirection 301) ; les pièces jointes privées passent par l'API.
4. Chaque push sur main : CI (tests) → images Docker (GHCR) → déploiement Ansible sur le VPS.

Figure 1 — Architecture de production

Choix structurants

- **Monolithe modulaire** : un seul déploiement, mais des modules isolés (Clean Architecture), chacun avec son schéma de base, ses contrats et ses événements. Un module peut être extrait en microservice plus tard sans réécriture.
- **Serveur unique, simple à opérer** : Caddy installé sur l'hôte (HTTPS automatique), API en conteneur, PostgreSQL natif, stockage objet externe.
- **Application mobile unique** pour iOS et Android (Expo), avec une couche de données interchangeable (API réelle ou démonstration hors ligne).
- **Tout est automatisé** : tests, images, déploiement, configuration Caddy et site public par la CI/CD.

3. Structure du dépôt

```
BeninConnect/
├── server/                          # Backend
│   ├── src/BeninConnect.Api/          # API (projet unique)
│   │   ├── Program.cs                 # composition, pipeline, --migrate
│   │   ├── Host/                      # Production.cs, DatabaseStartup, modules, démo
│   │   ├── Shared/                    # noyau commun : Domain, Application, Infrastructure, Api
│   │   ├── Modules/                   # 9 modules (voir §4)
│   │   ├── appsettings*.json · Dockerfile
│   ├── tests/BeninConnect.Api.Tests/  # intégration (PostgreSQL/MinIO), architecture, domaine
│   ├── docker-compose.yml             # développement local
│   ├── docker-compose.prod.yml        # production : api, backup, media-backup
│   ├── Caddyfile                      # api.monbenin.com + monbenin.com
│   ├── .env.example                   # modèle de configuration (sans secrets)
│   └── tools/media-to-s3.sh           # copie disque → R2
├── mobile-expo/                     # Application mobile
│   ├── src/app/                       # écrans (expo-router) : (auth), (app), onboarding
│   ├── src/components/                # UI, cartes, pièces jointes, vidéo, réactions
│   ├── src/core/                      # config, session, push, thème, photos, images…
│   ├── src/data/                      # contrat DataSource, client API, mock
│   ├── src/features/                  # requêtes, temps réel, signalement, enregistrement vocal
│   ├── plugins/                       # plugin Expo (nom Android)
│   ├── store/ios · store/android      # visuels des stores
│   ├── __tests__/ · app.json · eas.json · google-services.json
├── admin-console/                   # Console d'administration (React + Vite)
├── site/                            # Site public statique
├── ansible/                         # Rôles common, docker, app, web, security
├── .github/workflows/               # ci.yml, deploy.yml
├── docs/                            # mise en prod, CI/CD, backend, notifications…
├── tools/                           # ef-migrations.sh, scripts
├── src/ · tests/                      # prototype Avalonia (historique)
└── Directory.Packages.props · global.json · BeninConnect.sln
```

Structure d'un module backend

```
Modules/Messaging/
├── Contracts/        # seule partie visible des autres modules : DTO, interfaces, événements
├── Domain/           # entités et règles : Conversation, Message, MessageAttachment
├── Application/      # commandes, requêtes, gestionnaires, abstractions (IMessagingDb, IChatNotifier)
├── Infrastructure/   # EF Core (DbContext, migrations), SignalR (ChatHub), services
└── Api/              # MessagingModule : services et routes HTTP
```

4. Backend : monolithe modulaire

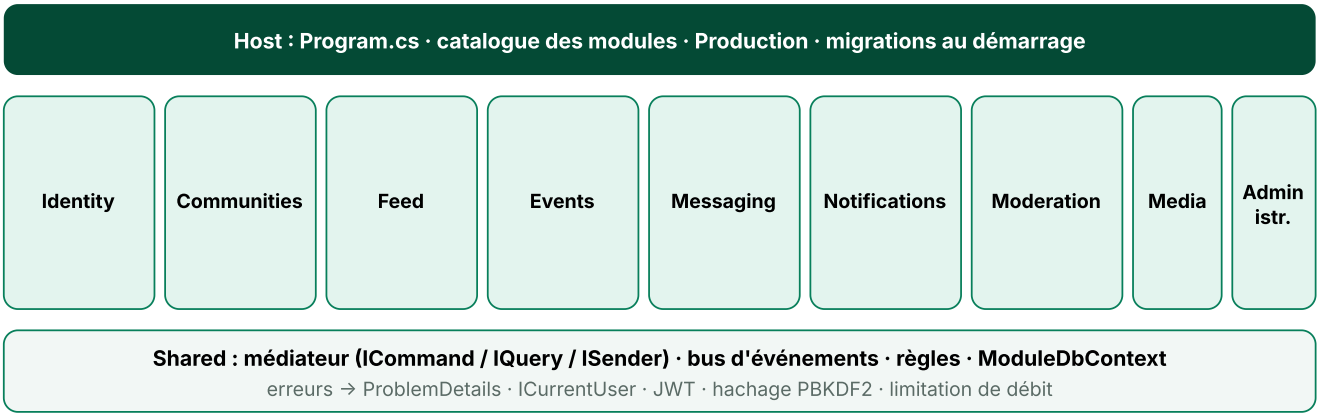


Figure 2 — Modules et noyau commun ; chaque module contient les couches Contracts, Domain, Application, Infrastructure, Api

Principes

- **Clean Architecture par module** : le domaine ne dépend de rien ; l'application dépend du domaine et d'abstractions ; l'infrastructure les implémente ; l'API ne fait que composer et exposer.
- **Frontières vérifiées par des tests** : `ArchitectureTests` analyse les sources et échoue si un module référence autre chose que les *Contracts* d'un autre module.
- **Une base logique par module** : chaque module a son `DbContext`, son schéma PostgreSQL (`identity`, `communities`, `feed`, `events`, `messaging`, `notifications`, `moderation`, `administration`) et sa table d'historique de migrations. Aucune clé étrangère entre modules : on référence un identifiant.
- **Médiateur maison** : commandes et requêtes (`ICommand<T>`, `IQuery<T>`), validation déclarative (`IValidatable`), gestionnaires enregistrés par espace de noms.
- **Erreurs normalisées** : `DomainException` → 400, `NotFoundException` → 404, `ForbiddenException` → 403, `ConflictException` → 409, toutes au format ProblemDetails.
- **Migrations** appliquées au démarrage (`Database:InitializeOnStartup`) ou par `dotnet BeninConnect.Api.dll --migrate`. Création : `tools/ef-migrations.sh <Nom> <Module>`.

Paquets principaux

Paquet	Version	Usage
Microsoft.EntityFrameworkCore / Npgsql.EntityFrameworkCore.PostgreSQL	10.0.0	Persistance, migrations
Microsoft.AspNetCore.Authentication.JwtBearer	10.0.0	Jetons membres et administrateurs
Microsoft.AspNetCore.SignalR (framework)	10	Messagerie en temps réel
AWSSDK.S3	4.0.14	Stockage objet (Cloudflare R2, MinIO…)
Testcontainers.PostgreSql / Minio, xUnit, Mvc.Testing	4.4.0 / 2.9.2	Tests d'intégration

Les versions sont centralisées dans `Directory.Packages.props` (gestion centrale des paquets NuGet).

5. Modules métier et API

Module	Responsabilité	Routes principales	Migrations
Identity	Comptes, vérification email, mots de passe, profils, suspension, suppression de compte	`/api/auth/{login, register, verify-email, resend-code, forgot-password, reset-password, me}` · `/api/users (recherche, profil, PUT /me, POST /me/delete)` · `/api/admin/members`	InitialCreate
Communities	Communautés publiques/privées, adhésions, demandes	`/api/communities (liste, détail, membres, join, membership, join-request)` · `/api/admin/communities`	InitialCreate
Feed	Publications (photo ou vidéo), j'aime, partages, commentaires, réactions, modération	`/api/feed` · `/api/posts (GET, POST, PUT, DELETE, like, share, comments, reactions)` · `/api/admin/posts (hide, unhide, delete)`	InitialCreate, AddPostEditedAt, AddPostVideo
Events	Événements, inscriptions, capacité, filtres	`/api/events (liste, catégories, villes, détail, participation)` · `/api/admin/events`	InitialCreate
Messaging	Conversations privées, messages, réactions, lecture, modification/suppression, pièces jointes, temps réel	`/api/conversations (direct, messages, attachments, PUT/DELETE message, read)` · `/api/messages/{id}/attachment` · `hub /hubs/chat`	InitialCreate, AddMessageEditDelete, AddMessageAttachments
Notifications	Centre de notifications, appareils push, envoi Expo	`/api/notifications (liste, unread-count, read, read-all, devices, devices/unregister)`	InitialCreate, AddPushDevices
Moderation	Signalements des membres, traitement par la modération	`POST /api/reports` · `/api/admin/reports (liste, resolve)`	InitialCreate
Media	Photos et vidéos publiques, pièces jointes privées, stockage local ou S3	`POST /api/media` · `POST /api/media/video` · `GET|HEAD /uploads/{nom}`	(aucune table)
Administration	Comptes administrateurs, permissions, statistiques, exports, journal d'audit, système	`/api/admin/{auth, stats, exports, admins, audit, system}`	InitialCreate

> Toutes les routes membres exigent un jeton (sauf inscription et connexion). Les routes `/api/admin/*` utilisent un schéma d'authentification distinct et des permissions par politique (`admin:{Permission}`). L'endpoint de réponse simulée (`demo-reply`) et la réinitialisation de démo ne sont exposés qu'en développement.

6. Communication entre modules

Les modules ne s'appellent jamais directement. Deux mécanismes, tous deux définis dans les *Contracts* :

- **Interfaces de module** (lecture synchrone) : `IIdentityModule` (noms, existence, suspension), `ICommunitiesModule` (adhésions), `IFeedModule`, `IMediaModule` (enregistrer/ouvrir une pièce jointe), `IModerationModule` …
- **Événements d'intégration** (bus en mémoire, `IEventBus`) : un module publie un fait, les autres réagissent.

Événement	Émetteur	Réactions
`MemberVerified`	Identity	Notifications : message de bienvenue
`MemberDeleted`	Identity	Feed, Communities, Events, Messaging, Notifications, Moderation : effacement de toutes les données du membre
`PostLiked` · `PostCommented` · `CommentReacted`	Feed	Notifications : notification + push à l'auteur
`CommunityJoined`	Communities	Notifications
`CommunityDeleted`	Communities	Feed et Events : publications et événements détachés de la communauté
`MessageSent`	Messaging	Notifications : notification + push aux destinataires
`ImagesReleased`	Identity, Feed, Communities, Events	Media : effacement des photos/vidéos devenues inutiles
`AttachmentsReleased`	Messaging	Media : effacement des pièces jointes privées
`NotificationRequested`	tout module	Notifications : notification générique

Exemple : suppression d'un compte

1. `POST /api/users/me/delete` avec le mot de passe ; Identity vérifie le mot de passe et supprime l'utilisateur.
2. Identity publie `MemberDeleted` puis `ImagesReleased` (photo de profil et couverture).
3. Chaque module efface ses données : publications, commentaires et j'aime ; communautés et événements créés, adhésions et inscriptions ; conversations et messages ; notifications et appareils ; signalements envoyés.
4. Media efface les fichiers publiés (`uploads/`) et les pièces jointes privées (`private/`).

7. Sécurité

Sujet	Mise en œuvre
Authentification	JWT signés (clé ≥ 32 caractères, `JWT_SIGNING_KEY`) ; schéma membre par défaut et schéma administrateur distinct. Un membre suspendu voit ses jetons refusés immédiatement (vérification à chaque requête).
Mots de passe	Hachage PBKDF2 ; codes de vérification et de réinitialisation à durée limitée et à tentatives limitées.
Autorisations	Contrôles métier dans les gestionnaires : seul l'auteur modifie ou supprime sa publication, seul l'expéditeur modifie (15 min) ou supprime son message, seuls les participants lisent une conversation et ses pièces jointes (404 sinon).
Limitation de débit	Politiques `Auth` (connexion, inscription, codes, suppression de compte) et `Uploads` (fichiers) en production.
Garde-fous de production	`Production.Validate` refuse de démarrer avec une configuration dangereuse : emails en mode journal, base sans SSL (sauf `DB_ALLOW_INSECURE` pour une base locale), clé JWT faible, S3 incomplet…
Fichiers	Format reconnu sur le contenu (signature binaire), jamais sur le nom ; noms aléatoires de 128 bits ; tailles maximales (photo 10 Mo, pièce jointe 20 Mo, vidéo 60 Mo). Pièces jointes dans un stockage privé jamais servi directement.
Jeton dans l'URL	Accepté uniquement pour `/hubs` (WebSocket) et `/api/messages/{id}/attachment` (image, audio, PDF ouverts par le téléphone). Caddy masque `access_token` dans ses journaux.
Transport	HTTPS partout (Caddy, Let's Encrypt), HSTS, en-têtes de sécurité, API liée à `127.0.0.1` ; pare-feu UFW (22, 80, 443) géré par Ansible.
Secrets	`.env` déposé à la main sur le serveur (droits 600), jamais dans git ; secrets GitHub (SSH, registre) ; identifiants Apple/Google gérés par EAS.
Contenu des membres	Signalement dans l'app, modération dans la console, conditions d'utilisation à tolérance zéro, suppression du compte dans l'app.

8. Temps réel et notifications

SignalR — `/hubs/chat`

Le hub est en lecture seule pour le client : le serveur pousse, le client n'appelle aucune méthode. L'utilisateur est identifié par la revendication `sub` du jeton (`MemberIdProvider`) ; les messages sont envoyés à tous les participants connectés, sur tous leurs appareils.

Événement	Déclencheur
`messageCreated(message)`	Nouveau message (texte ou pièce jointe)
`messageUpdated(message)`	Modification, suppression ou réaction
`conversationRead(conversationId, readerId)`	Le destinataire a lu les messages

Côté application (`src/features/realtime.ts`) : connexion WebSocket ouverte tant que le membre est connecté, reconnexion automatique, rattrapage des messages au retour au premier plan, mise à jour directe des caches TanStack Query. Une seule instance d'API : aucun fond de panier (Redis) n'est nécessaire.

Notifications push

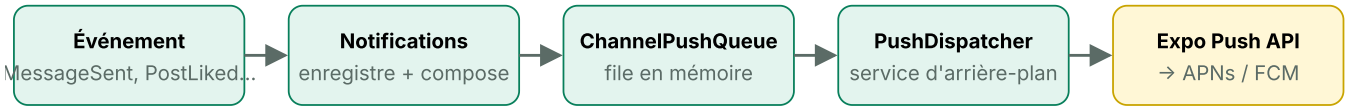


Figure 3 — Chaîne d'envoi des notifications push

- Enregistrement de l'appareil : `POST /api/notifications/devices` (jeton `ExponentPushToken[…]` , plateforme, catégories désactivées). Désinscription à la déconnexion.
- Envoi par lots de 100, badge = nombre de notifications non lues, données de navigation (`target` , `targetId`) ; les jetons invalides (`DeviceNotRegistered`) sont supprimés.
- iOS : clé APNs gérée par EAS. Android : projet Firebase, `google-services.json` et clé FCM V1 chargée dans EAS.
- Option : `EXPO_ACCESS_TOKEN` pour sécuriser l'envoi ; `Push:Provider=log` en test.

9. Médias et stockage

Zone	Contenu	Accès
Public `uploads/`	Photos de profil, couvertures, photos et vidéos des publications, miniatures	`https://api.monbenin.com/uploads/<nom>` ; en S3 avec domaine public : redirection 301 vers l'URL R2 (CDN)
Privé `private/`	Pièces jointes des discussions : photos, PDF, messages vocaux	Uniquement `GET /api/messages/{id}/attachment`, après vérification que le demandeur participe à la conversation

Formats et limites

Type	Formats acceptés (détection sur le contenu)	Taille max	Endpoint
Photo	JPEG, PNG, WebP, HEIC	10 Mo	`POST /api/media`
Vidéo	MP4, MOV (60 s, H.264 720p sur iPhone)	60 Mo	`POST /api/media/video`
Pièce jointe	Photos, PDF, audio (M4A, MP3, AAC, WAV, OGG, CAF)	20 Mo	`POST /api/conversations/{id}/attachments`

Fournisseurs

- **Local** (`MEDIA_PROVIDER=local`) : volume Docker `media` (`uploads/` et `private/`), servi par l'API (fichiers statiques, requêtes partielles natives), sauvegardé chaque nuit.
- **S3** (`MEDIA_PROVIDER=s3`), **en service** : Cloudflare R2, bucket `beninconnect-media` (photos et vidéos, URL publique `pub-….r2.dev`) et bucket `beninconnect-private` (pièces jointes, jamais public). Sans URL publique, l'API relaie les fichiers avec gestion des requêtes `Range`, indispensable au lecteur vidéo d'iOS.
- **Migration** : `server/tools/media-to-s3.sh` copie le volume vers les buckets (sans suppression, relançable). Les chemins en base (`uploads/<nom>`) ne changent pas.
- **Nettoyage** : un fichier remplacé ou dont le propriétaire est supprimé est effacé du stockage (événements `ImagesReleased` et `AttachmentsReleased`).

> L'URL `r2.dev` est prévue par Cloudflare pour le développement (débit limité, sans cache). Évolution conseillée : déplacer le DNS de `monbenin.com` chez Cloudflare et brancher le domaine `media.monbenin.com` sur le bucket public.

10. Application mobile

Pile technique

- Expo SDK 58, React Native 0.88, React 19.3, TypeScript
- Navigation : expo-router (groupes `(auth)` et `(app)`, onglets)
- État serveur : TanStack Query ; état local : Zustand
- Temps réel : @microsoft/signalr
- Médias : expo-image, expo-image-picker, expo-video, expo-video-thumbnails, expo-audio, expo-document-picker, expo-web-browser
- Notifications : expo-notifications, expo-device
- Stockage : expo-secure-store (jeton), AsyncStorage (préférences)

Identité de l'application

Nom affiché	BéninConnect
Identifiant	`com.hbatech.beninconnect.rn`
Version	1.0.0 (numéros de build gérés par EAS)
Appareils	iPhone (iPad désactivé), Android
Projet EAS	`@berihectors-team/beninconnect`
API	`https://api.monbenin.com`

Organisation du code

Dossier	Contenu
`src/app/`	Écrans : connexion, inscription, vérification, accueil, communautés, publication (création/modification), messages, conversation, événements, profil, paramètres, suppression de compte
`src/data/`	`repositories.ts` (contrat `DataSource`), `api/apiDataSource.ts` (client HTTP, envoi XHR des fichiers), `mock/` (démonstration hors ligne), `types.ts`
`src/features/`	`queries.ts` (requêtes et mutations, mises à jour des caches), `realtime.ts`, `report.ts`, `voice.ts`
`src/core/`	`config.ts` (URL de production, `DEV_TOOLS`), session, push, photos/vidéos, images, thème, préférences
`src/components/`	Composants d'interface, cartes (publication, message), pièces jointes, lecteur vidéo, réactions

Comportements notables

- **Production verrouillée** : hors développement (`__DEV__`) ou sans `EXPO_PUBLIC_DEV_TOOLS=1` , l'application utilise toujours l'API de production et masque les réglages de source de données.
- **Envoi de fichiers** par `XMLHttpRequest` (le `fetch` d'Expo 58 ne lit pas les fichiers `{ uri }`), avec progression pour la vidéo.
- **Autorisations iOS** : photos, appareil photo, micro (messages vocaux), notifications ; textes en français dans `app.json` .
- **Android** : icône adaptative dédiée, nom avec accent via `plugins/withAndroidAppName.js` , FCM via `google-services.json` .

11. Console d'administration et site public

Console d'administration (`admin-console/`)

Application React 19 + Vite + TanStack Query, authentifiée par le schéma administrateur de l'API, avec permissions par rôle.

Page	Rôle
Tableau de bord	Statistiques (membres, contenus, activité, pays)
Signalements	File de modération : traiter, masquer, supprimer, suspendre
Membres	Recherche, fiche, suspension, réactivation
Publications, Communautés, Événements	Consultation et modération des contenus
Administrateurs, Compte	Gestion des comptes et des permissions, mot de passe
Journal d'audit	Traçabilité des actions de modération
Exports, Système, Erreurs	Exports CSV, état du serveur et du stockage

> Le conteneur de la console a été retiré du déploiement de production (Traefik remplacé par Caddy). Elle peut être servie en statique par Caddy (par ex. `admin.monbenin.com`) le moment venu.

Site public (`site/`)

Page	Usage
`/`	Présentation, fonctionnalités, captures
`/support/`	Contact et questions fréquentes (URL de support des stores)
`/confidentialite/`	Politique de confidentialité (RGPD)
`/conditions/`	Conditions d'utilisation, tolérance zéro, modération
`/mentions-legales/`	Éditeur, hébergeur, propriété intellectuelle
`/supprimer-compte/`	Procédure de suppression (URL exigée par Google Play)

Pages statiques responsive (mode sombre automatique), 404 personnalisée, `robots.txt` , `sitemap.xml` . Déployées par la CI/CD dans `/var/www/monbenin` .

12. Configuration

Variables du serveur (`/opt/beninconnect/server/.env`)

Modèle : `server/.env.example`. Le fichier réel n'est jamais versionné ; Ansible vérifie sa présence et ses droits (600).

Variable	Rôle
`API_PORT`	Port local de l'API (8080), lié à 127.0.0.1
`DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD`	PostgreSQL (hôte : `host.docker.internal`, base native du VPS)
`DB_SSLMODE, DB_ALLOW_INSECURE, DB_POOL_SIZE`	SSL de la base, exception pour une base locale, taille du pool
`JWT_SIGNING_KEY`	Clé de signature des jetons (`openssl rand -base64 48`)
`EMAIL_PROVIDER, RESEND_API_KEY, EMAIL_FROM, EMAIL_FROM_NAME`	Emails via Resend (ou SMTP_*)
`EXPO_ACCESS_TOKEN`	Facultatif : sécurisation de l'envoi des push
`MEDIA_PROVIDER`	`local` ou `s3` (actuellement `s3`)
`S3_SERVICE_URL, S3_REGION, S3_BUCKET, S3_PRIVATE_BUCKET`	Cloudflare R2 : point d'accès, région `auto`, buckets public et privé
`S3_ACCESS_KEY, S3_SECRET_KEY`	Clé R2 limitée aux deux buckets (lecture/écriture)
`S3_PUBLIC_BASE_URL, S3_CREATE_BUCKET`	URL publique du bucket média ; création automatique (MinIO uniquement)
`CORS_ORIGIN_1, CORS_ORIGIN_2`	Origines autorisées (console web)
`ADMIN_BOOTSTRAP_*`	Premier administrateur créé au démarrage

Application mobile

Fichier	Contenu
`app.json`	Nom, identifiants, icônes, autorisations, plugins (image-picker, notifications, audio, splash, nom Android), projet EAS
`eas.json`	Profils `preview` et `production` (incrément automatique, .aab Android) ; envoi App Store (`ascAppId`) et Google Play (test interne, brouillon)
`google-services.json`	Projet Firebase (FCM) ; non secret
`.env.local`	`EXPO_PUBLIC_API_URL` pour pointer vers une API locale en développement

13. Infrastructure et déploiement

Élément	Détail
Serveur	VPS OVH, Ubuntu, IP 51.255.40.214, utilisateur `ubuntu`, dossier `/opt/beninconnect`
Domaines	`api.monbenin.com` (API), `monbenin.com` et `www` (site) ; DNS chez OVH ; emails OVH (MX) et Resend (DKIM)
Caddy	Installé sur l'hôte, configuration `/etc/caddy/sites/beninconnect.caddy` (importée par `/etc/caddy/Caddyfile`) ; HTTPS automatique, HSTS, corps de requête jusqu'à 70 Mo, journaux JSON avec masquage du jeton
Conteneurs (`docker-compose.prod.yml`)	`api` (image GHCR `beninconnect-api:sha-…`, 127.0.0.1:8080, volume `media`), `backup` (sauvegarde PostgreSQL quotidienne), `media-backup` (archive du volume média)
PostgreSQL	Installé nativement sur le VPS ; accès des conteneurs via `host.docker.internal` (règle UFW 172.16.0.0/12, `listen_addresses` et `pg_hba.conf` adaptés)
Stockage objet	Cloudflare R2 (photos, vidéos, pièces jointes)

Ansible (`ansible/`)

Rôle	Tâches
`common`	Paquets, fuseau horaire, swap, utilisateur de déploiement et clés SSH
`docker`	Installation de Docker et du plugin compose
`app`	Fichier compose, vérification du `.env`, connexion au registre GHCR, déploiement de l'image, contrôle de santé (journaux en cas d'échec)
`web`	Caddy : dossiers, import des sites, configuration validée par `caddy validate` puis rechargement ; synchronisation du site public
`security`	Pare-feu UFW (SSH, 80, 443), durcissement SSH

`site.yml` installe un serveur complet ; `deploy.yml` (rôles `app` et `web`) est exécuté à chaque déploiement.

14. CI/CD

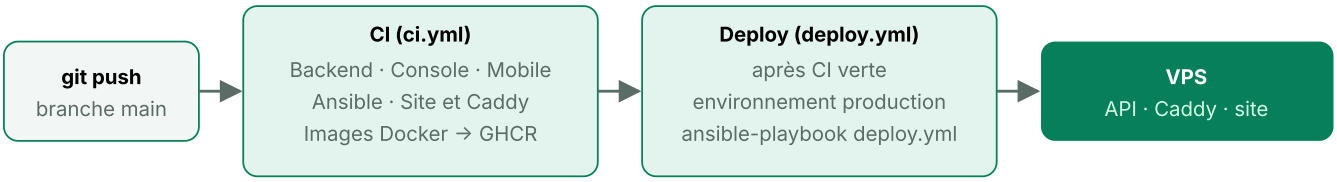


Figure 4 — Chaîne de livraison continue

Job	Vérifications
Backend (.NET 10)	Build Release, tests d'intégration (PostgreSQL et MinIO éphémères via Testcontainers), architecture modulaire, migrations à jour
Console admin	oxlint, vitest, `tsc`, build Vite
Application mobile	`tsc --noEmit`, Jest
Ansible	Vérification syntaxique des playbooks, ansible-lint
Site public et Caddy	`caddy validate` du Caddyfile, contrôle des liens internes du site
Images (main)	Image `ghcr.io/<propriétaire>/beninconnect-api`, étiquettes `sha-<commit>` et `latest`

Secrets GitHub (environnement `production`)

- `SSH_PRIVATE_KEY` : clé de déploiement (clé publique autorisée sur le VPS)
- `SSH_KNOWN_HOSTS` : empreinte du serveur (`ssh-keyscan`)
- `GHR_TOKEN` : lecture des images sur GHCR par le serveur

Déploiement manuel d'une version précise : Actions › Deploy › Run workflow avec une étiquette (`sha-1a2b3c4`), ce qui permet aussi de revenir en arrière.

15. Tests et qualité

Suite	Couverture
Backend — 66 tests xUnit	Architecture frontières entre modules, couches Domaine règles pures (j'aime, adhésion, fenêtre de modification des messages...) Intégration API réelle sur PostgreSQL : authentification, contrats, administration, push, emails, suppression de compte, temps réel, pièces jointes, vidéo S3 stockage MinIO : photos, vidéo avec requêtes partielles, pièces jointes privées (en CI) Production garde-fous de configuration
Mobile — 45 tests Jest	Source de démonstration et client API : photos, suppression, signalement, modifications, pièces jointes, vidéo, formats, règles, réactions
Console — vitest	Utilitaires (format, permissions, client API)

Exécution locale

```
# Backend (Docker requis pour Testcontainers, ou BC_TEST_POSTGRES=« Host=…;Username=…;Password=… »)
dotnet test server/tests/BeninConnect.Api.Tests

# Mobile
cd mobile-expo && npx tsc --noEmit && npx jest

# Nouvelle migration pour un module
tools/ef-migrations.sh AddQuelqueChose Feed
```

16. Publication sur les stores

Commandes

```
cd mobile-expo
eas build -p all --profile production     # builds iOS (.ipa) et Android (.aab) sur les serveurs EAS
eas submit -p ios --latest                # App Store Connect (ascAppId 6832112483)
eas submit -p android --latest            # Google Play, test interne (google-play-service-account.json)
```

App Store

- Fiche en français : nom, sous-titre, description, mots-clés, texte promotionnel
- Captures iPhone 1206 × 2622 et bannière d'en-tête (`store/ios/`)
- URL de support et de confidentialité sur `monbenin.com`
- App Privacy : coordonnées, contenus, photos/vidéos, audio, identifiants
- DAC7, copyright, compte de test pour la relecture
- Clé APNs (push) gérée par EAS

Google Play

- Captures 1080 × 1920, icône 512, bannière 1024 × 500 (`store/android/`)
- API Google Play Developer activée, compte de service invité
- Premier envoi manuel du .aab, puis EAS
- Sécurité des données, classification, public cible
- URL de suppression de compte : `/supprimer-compte/`
- Comptes personnels récents : test fermé (12 testeurs, 14 jours) avant la production

17. Points restants et évolutions

Priorité	Sujet	Détail
Haute	Blocage d'utilisateur	Ne plus voir les contenus ni recevoir les messages d'un membre (attendu par Apple, règle 1.2)
Haute	Domaine média	DNS de `monbenin.com` chez Cloudflare, puis `media.monbenin.com` sur le bucket R2 (remplace l'URL `r2.dev` limitée)
Moyenne	Sauvegardes hors site	Copier les sauvegardes PostgreSQL du VPS vers R2 ou un autre stockage
Moyenne	Mentions légales	Ajouter l'adresse du siège et le numéro RCCM de HBA Tech et Trade
Moyenne	Console d'administration	La publier (statique derrière Caddy, ex. `admin.monbenin.com`)
Moyenne	Vidéo sur Android	Compression avant envoi (pas de conversion automatique comme sur iPhone)
Basse	Dépendances	Mettre à jour les paquets signalés vulnérables (Microsoft.OpenApi, System.Security.Cryptography.Pkcs, SSH.NET côté tests)
Basse	Langues	Anglais dans l'application et sur le site
Basse	Montée en charge	Plusieurs instances de l'API : fond de panier SignalR (Redis) et file de push partagée

Documentation du dépôt

Fichier	Sujet
`docs/MISE_EN_PROD.md`	Base de données, médias et S3, emails, migrations, Caddy
`docs/BACKEND_MODULAIRE.md`	Règles du monolithe modulaire
`docs/CI_CD.md`	Workflows, secrets, site public, retour arrière
`docs/NOTIFICATIONS_PUSH.md`	APNs, FCM, Expo
`ansible/README.md`	Installation et déploiement du serveur